

La prochaine frontière : IA et code

Christian Jauvin, professeur

Daniel Lemire, professeur

Université du Québec (TÉLUQ)

Montréal 🇨🇦

blogs: <https://cjauvin.github.io> <https://lemire.me>

X: [@ChristianJauvin](#) [@lemire](#)

GitHub: <https://github.com/cjauvin> <https://github.com/lemire/>



Daniel Lemire ✓
@lemire

Boost



What fraction of your code is typed by hand (no AI)?



1,023 votes · Final results

7:19 PM · Mar 15, 2026 · **13.5K** Views

~ claude



Claude Code v2.1.144

Opus 4.7 (1M context) · Claude Max
/Users/dlemire

> Try "how do I log an error?"

▶▶ auto mode on (shift+tab to cycle) · ← for agents

Markdown

Introduction à Java (exemple)

Nous pouvons définir une variable en Java ainsi.

```
```java  
int x = 1;
```

Notez les différents éléments de la syntaxe.

- Type
- Nom de variable
- Assignment d'une valeur

---

# Skills (compétences)

## # Data Plot Skill

Given a query, find reliable online data, download it, and produce bilingual (French/English) matplotlib plots.

## ## Working directory

All output goes in `~/myplots/<slug>/` where <slug> is a short kebab-case name for the query.`

1. Ensure `~/myplots` exists (mkdir -p ~/myplots`)`
2. Create the subdirectory for this query, e.g. `~/myplots/canada-population-2024/``
3. All scripts, data, README, and PNGs go in that subdirectory

## ## Data sourcing

- Prefer official/government sources: Statistics Canada, Statistique Canada, data.gouv, U.S. Census, Eurostat, World Bank, OECD, etc.
- Prefer machine-readable formats in this order: **\*\*CSV > JSON > HTML table\*\***
- Use WebSearch + WebFetch to locate the dataset URL. Verify the URL returns the expected format before scripting.
- Record the exact source URL and access date for the README.

## ## Python environment

Always use `uv` for dependencies. Initialize the project in the subdirectory:`

```
```bash
cd ~/myplots/<slug>
uv init --no-readme --no-workspace
uv add pandas matplotlib requests
```

Utilisation

```
/plotdata I want the fertility rate per canadian province along with the percentage of woman in each province with a university degree.
```

What the chart shows – one labelled dot per province: x-axis is the share of women aged 25–64 with a bachelor's degree or higher (2021 Census), y-axis is the total fertility rate (2024), with a least-squares trend line.

Key findings:

- The two metrics are only weakly correlated across provinces ($r \approx -0.14$) – more university-educated women does *not* clearly track higher or lower fertility.
- **Saskatchewan** (1.58 children/woman) and **Manitoba** (1.50) lead on fertility with mid-range education (~30–33%).
- **British Columbia** sits at the bottom for fertility (1.02) despite high education (38.2%); **Newfoundland and Labrador** is also low (1.12) with the lowest education share (24.3%).
- **Ontario** has the highest share of university-educated women (39.6%) and below-average fertility (1.21).

Data sources:

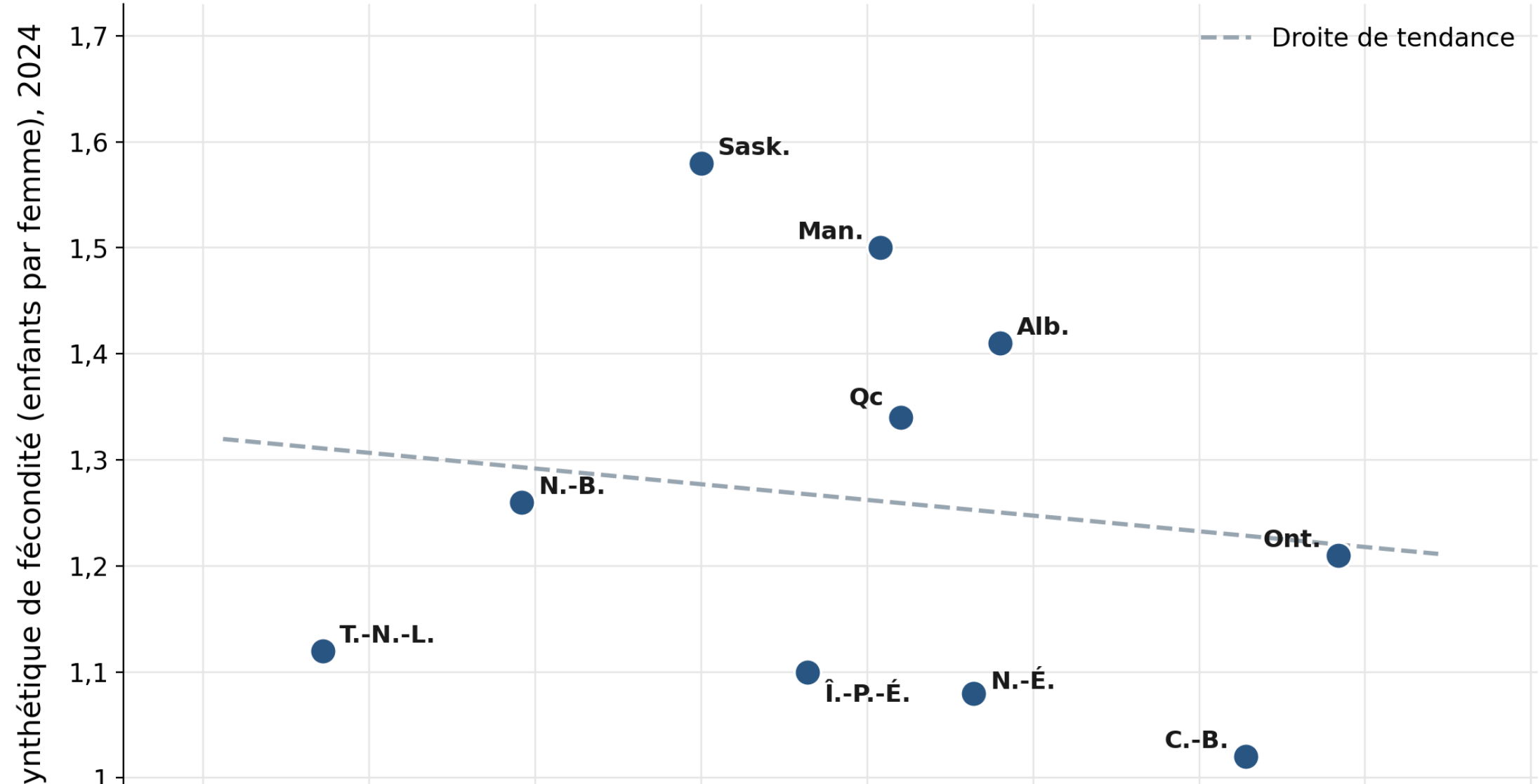
- Fertility: [Statistics Canada, Table 13-10-0418-01](#) (total fertility rate, 2024)
- Education: [Statistics Canada, Table 98-10-0384-01](#) (2021 Census; women 25–64 with a bachelor's degree or higher)

Two caveats worth noting: the metrics come from different years (2024 vs. the 2021 Census, the latest education table available), and the chart is a structural comparison across provinces, not evidence of causation. Territories were excluded since you asked for provinces – let me know if you'd like them added, or a different age band/degree definition.

* Churned for 5m 6s

* **recap:** Goal was bilingual scatter plots of fertility rate vs. women's university education by Canadian province. Both plots, scripts, data, and README are complete in `~/myplots/canada-fertility-education/`. Next action: optionally add territories or adjust the age band/degree definition if you want changes. (disable recaps in `/config`)

Fécondité et scolarité universitaire des femmes dans les provinces canadiennes



reordered so each one builds on the last:

MCP — The foundation. Before agents can do anything interesting, they need a standard way to talk to tools and data. Start here because everything downstream assumes the model can reach outside itself.

Google Calendar, Drive, Slack, GitHub, Git, Postgres,

Hooks

AI Skills — Now that the model can call tools, how do you package capabilities so it knows when and how to use them? Skills are the layer above raw tool access — instructions, examples, and scaffolding that turn a tool into a reliable behavior.

Language Servers — A concrete, killer example of skill-shaped tool integration: give the agent the same code intelligence your IDE has. This makes the abstract idea of "good tooling" visceral — go-to-definition, type info, diagnostics, all available to the model.

Hooks — Once your agent is doing real work, you need control. Hooks are how you

Plan de la présentation

- L'état actuel de l'IA générative pour le code
- Comment fonctionnent les grands modèles de langage (LLM)
- Outils d'assistance au codage
- Qualité et performance du code généré
- Impacts sur la profession
- Ce qui vient ensuite

La révolution en cours

- Les LLM génèrent du code depuis ~2021 (Codex, Copilot)
- En 2026, les assistants IA sont omniprésents
- Plus de 70% des développeurs utilisent un assistant IA
- La question n'est plus « si » mais « comment »

Qu'est-ce qu'un grand modèle de langage?

- Réseau de neurones entraîné sur d'immenses corpus de texte
- Prédit le prochain jeton (token) dans une séquence
- Le code est du texte : les LLM s'y appliquent naturellement
- Pas de « compréhension » au sens humain, mais une capacité de synthèse remarquable

L'entraînement en bref

- Phase 1 : pré-entraînement sur des milliards de lignes de code
- Phase 2 : fine-tuning avec des exemples de haute qualité
- Phase 3 : RLHF — apprentissage par rétroaction humaine
- Résultat : un modèle qui produit du code syntaxiquement correct la majorité du temps

Les outils d'aujourd'hui

Outil	Approche
GitHub Copilot	Complétion en ligne dans l'éditeur
Claude Code	Agent en ligne de commande
Cursor / Windsurf	IDE augmenté par IA
ChatGPT / Claude	Conversation interactive
Codex CLI	Exécution autonome de tâches

Démonstration typique

```
# Demande : trier une liste de dictionnaires par clé "score"
données = [{"nom": "Alice", "score": 88},
            {"nom": "Bob", "score": 95},
            {"nom": "Charlie", "score": 72}]

résultat = sorted(données, key=lambda x: x["score"],
                  reverse=True)
```

L'IA génère ce code en moins d'une seconde.

Mais est-ce **performant**?

Le piège de la facilité

- Le code généré compile et passe les tests de base
- Mais il peut être :
 - Sous-optimal en performance
 - Fragile face aux cas limites
 - Difficile à maintenir
- « Ça marche » $\neq$ « C'est bon »

Mesurer la performance : pourquoi c'est crucial

- Un programme 10× plus lent, c'est 10× plus de serveurs
- L'énergie consommée est proportionnelle au temps CPU
- La performance, c'est aussi de la durabilité
- Daniel Lemire : « La performance n'est pas un luxe, c'est une responsabilité »

Exemple : parsing JSON

- **simdjson** : ~3 Go/s sur un seul cœur
- Code naïf généré par IA : ~200 Mo/s
- Facteur **15×** de différence
- L'IA ne connaît pas les instructions SIMD spécialisées
- L'expertise humaine reste irremplaçable pour l'optimisation bas niveau

L'IA et les algorithmes

- Les LLM reproduisent les algorithmes courants (tri, recherche)
- Mais peinent avec :
 - L'optimisation vectorielle (SIMD)
 - Les structures de données non standard
 - Les compromis espace/temps subtils
- Les algorithmes nouveaux exigent encore une réflexion humaine

Bitmaps compressés : un cas d'étude

- Roaring Bitmaps : utilisé par Google, Apache, Uber
- Structure hybride : tableaux, bitmaps, runs
- L'IA peut *utiliser* la bibliothèque
- Mais elle ne pourrait pas *inventer* cette structure
- L'innovation algorithmique reste humaine

Ce que l'IA fait bien

- Écrire du code « boilerplate »
- Traduire entre langages
- Générer des tests unitaires
- Documenter du code existant
- Prototyper rapidement
- Expliquer du code inconnu

Ce que l'IA fait mal

- Reasonner sur la complexité algorithmique
- Optimiser pour le matériel spécifique
- Comprendre le contexte métier profond
- Gérer la concurrence et les conditions de course
- Garantir la sécurité (injections, failles)
- Maintenir la cohérence dans un grand projet

Le problème de l'hallucination

- Les LLM inventent des API qui n'existent pas
- Ils citent des bibliothèques fictives
- Le code peut sembler plausible mais être incorrect
- Vérification humaine **obligatoire**
- « Faire confiance, mais vérifier » — surtout vérifier

Transparence et honnêteté

- Christian Jauvin propose le concept de « Proof of Prompt »
- Idée : déclarer ouvertement quand l'IA a contribué
- Pas de honte à utiliser un outil
- Mais l'honnêteté intellectuelle exige la transparence
- Le code généré par IA devrait être étiqueté

L'impact sur les développeurs juniors

- Risque : apprendre à « prompter » au lieu de « programmer »
- L'IA masque la complexité sous-jacente
- Un développeur qui ne comprend pas le code qu'il utilise est fragile
- L'apprentissage des fondamentaux reste essentiel
- L'IA est un accélérateur, pas un substitut à la compétence

L'impact sur les développeurs seniors

- Productivité accrue pour les tâches répétitives
- Plus de temps pour l'architecture et la réflexion
- Nouveau rôle : « réviseur de code IA »
- La valeur se déplace vers le jugement, pas l'exécution
- Les experts en performance sont plus précieux que jamais

Benchmarks : code humain vs code IA

Tâche	Humain expert	IA (2026)
Code correct	95%	85%
Code performant	80%	40%
Code sécuritaire	75%	50%
Code maintenable	85%	60%

Estimations basées sur la littérature récente

Le coût énergétique de l'IA

- Entraîner un LLM : des milliers de MWh
- Chaque requête de complétion consomme de l'énergie
- Paradoxe : l'IA peut générer du code inefficace qui consomme *plus* de ressources à l'exécution
- Il faut mesurer le coût total : génération + exécution

Les langages et l'IA

- L'IA est meilleure en Python et JavaScript (plus de données)
- Performance variable en C, C++, Rust
- Les langages moins courants (Zig, Nim) posent problème
- Biais vers les solutions « populaires », pas les « optimales »
- Le choix du langage influence la qualité de la sortie IA

Tests et validation

- L'IA peut générer des tests, mais :
 - Elle teste ce qu'elle « pense » être correct
 - Les cas limites sont souvent négligés
 - La couverture est superficielle
- Le test reste une discipline humaine
- Fuzzing et tests de propriété : compléments essentiels

Sécurité du code généré

- Les LLM reproduisent des patrons vulnérables (injection SQL, XSS)
- Ils ne comprennent pas les modèles de menace
- Le code généré doit passer par les mêmes revues de sécurité
- Ne jamais déployer du code IA sans audit
- La sécurité est un processus, pas une fonctionnalité

L'IA comme partenaire de revue de code

- Excellente pour détecter :
 - Le code mort
 - Les incohérences de style
 - Les bogues évidents
- Moins fiable pour :
 - Les problèmes d'architecture
 - Les failles de sécurité subtiles
 - La logique métier

L'avenir proche (2026–2028)

- Agents autonomes capables de résoudre des bogues complets
- Meilleure intégration avec les outils de build et CI/CD
- Modèles spécialisés par domaine (embarqué, web, données)
- Compréhension de projets entiers, pas juste de fichiers isolés
- Boucle de rétroaction : l'IA apprend du code en production

L'avenir lointain : spéculations

- Programmation en langage naturel comme interface primaire?
- Les langages de programmation deviennent-ils des « bytecode »?
- Le développeur comme « architecte d'intention »?
- Ou bien : les fondamentaux restent, l'IA accélère
- L'histoire de l'informatique suggère : les abstractions montent, mais le bas niveau ne disparaît jamais

Conseils pratiques

1. Utilisez l'IA pour le code jetable et le prototypage
2. Vérifiez **toujours** le code généré
3. Mesurez la performance avant de faire confiance
4. Investissez dans votre compréhension des fondamentaux
5. Soyez transparent sur l'utilisation de l'IA
6. Restez curieux : l'outil évolue rapidement

Ce qu'on ne vous dit pas assez

- L'IA ne remplace pas 10 ans d'expérience en une requête
- Les benchmarks marketing $\neq$ la réalité du terrain
- Le code le plus rapide est celui qu'on n'exécute pas
- Un bon algorithme bat toujours un mauvais algorithme optimisé par IA
- La pensée critique est votre compétence la plus précieuse

Conclusion

- L'IA transforme le développement logiciel — c'est indéniable
- Mais elle ne remplace pas l'expertise, elle l'amplifie
- La performance, la sécurité, le jugement restent humains
- Adoptons ces outils avec lucidité et rigueur
- La prochaine frontière, c'est la collaboration humain-IA

Questions?

Christian Jauvin — cjauvin.github.io

Daniel Lemire — lemire.me

Merci! 🇨🇦